TD5 — Polices, icônes et formulaires

Objectif : comprendre ce qui existe dans l'HTML, le cibler en CSS, puis observer le résultat.

1 Google Fonts

Importer une police externe
puis l'utiliser avec font-family.

2 Material Symbols

Utiliser une bibliothèque d'icônes
et changer le symbole par son nom.

3 Formulaires

Comprendre les types d'input, les sélecteurs et les états :checked / :focus.

Méthode pendant tout le TD : HTML → CSS → modifier UNE chose → enregistrer → observer.

Avant de commencer — ouvrez les 3 vues

Nous gardons les trois ouvertes pendant tout le TD.

1. tuto.html

Ce qui existe
(tags, attributs, classes, id)

→

2. tuto.css

Comment on cible
et comment on le présente

→

3. Navigateur

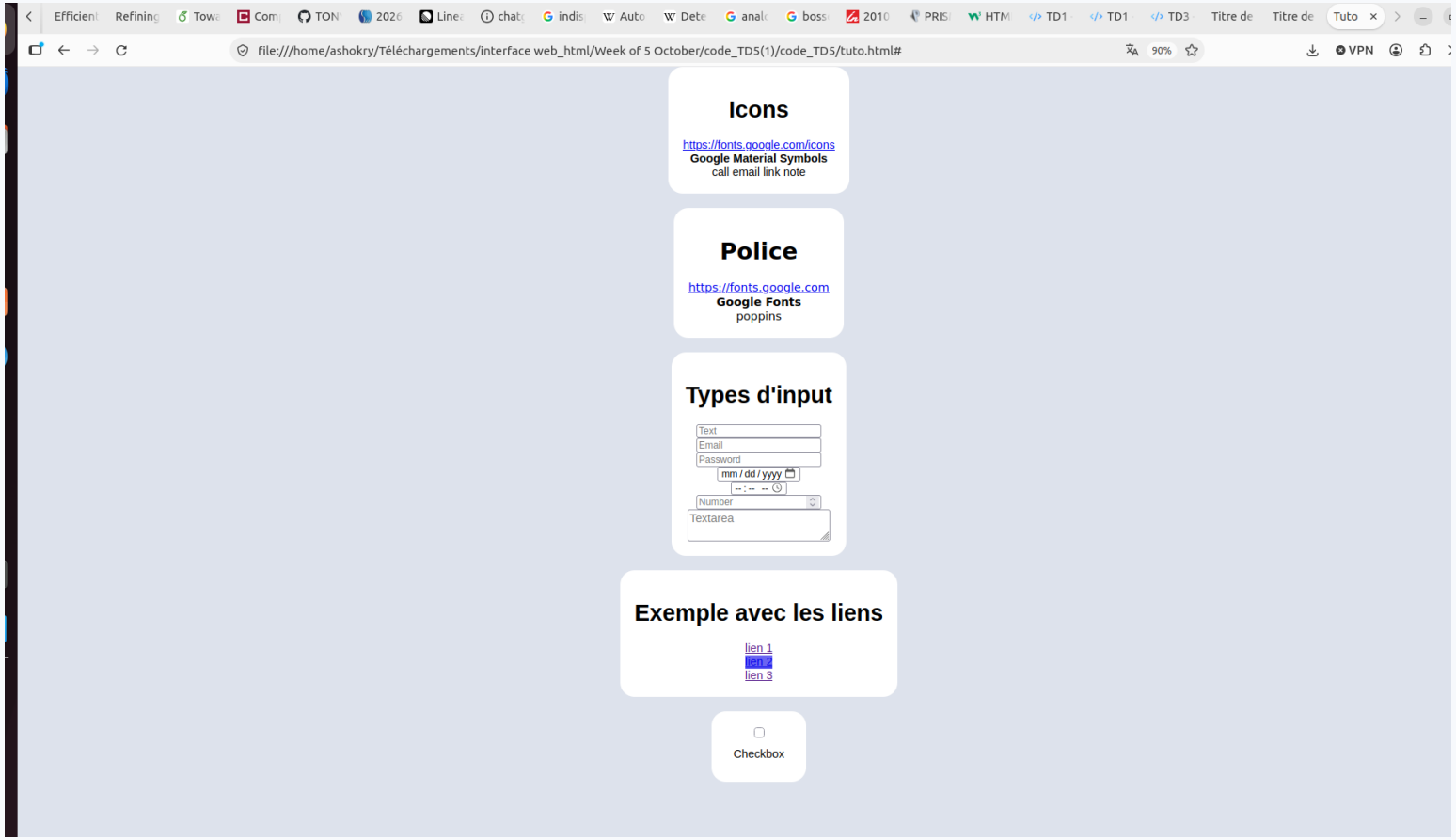
Ce qu’on observe
après Ctrl+S + actualiser

À FAIRE

Ouvrez tuto.html et tuto.css dans VS Code, puis ouvrez tuto.html dans le navigateur.

La page de support — 5 petits laboratoires

Chaque bloc sert à tester une notion avant l'exercice 1.



1. **Icons**
   bibliothèque d'icônes
2. **Police**
   Google Fonts
3. **Types d'input**
   contrôles HTML
4. **Liens**
   sélecteur par attribut
5. **Checkbox**
   état :checked

RÈGLE
On observe un bloc à la fois.

Dans <head> : ce qui est connu, nouveau… et à ne pas mémoriser

On classe les lignes avant de les utiliser.

DÉJÀ VU

```
<meta charset="UTF-8">
<meta name="viewport" ...>
<title>Tuto</title>
<link rel="stylesheet" href="tuto.css">
```

NOUVEAU AUJOURD'HUI

```
<link rel="stylesheet"
      href="...Material+Symbols...">

<link href="...family=Poppins..."
      rel="stylesheet">
```

TECHNIQUE

```
<link rel="preconnect"
      href="https://fonts.googleapis.com">
<link rel="preconnect"
      href="https://fonts.gstatic.com"
crossorigin>
```

À RETENIR

Les deux nouveaux <link> rendent disponibles Poppins et Material Symbols.

PAS À MÉMORISER

Les longues URL sont générées par Google. On les copie quand on en a besoin.

PAS L'OBJECTIF

preconnect optimise la connexion. Nous ne l'étudions pas dans ce TD.

Google Fonts — 2 actions seulement

1) rendre la police disponible ; 2) l’utiliser dans notre CSS.

HTML — DANS <head>

```
<link href="...family=Poppins..."
      rel="stylesheet">
```

CSS — UTILISER LA POLICE

```
.font * {
    font-family: "Poppins", sans-serif;
}
```

IMPORTER / RENDRE DISPONIBLE

UTILISER DANS LA ZONE CIBLÉE

SI JE VEUX UNE AUTRE POLICE

Dans CE support, Poppins est déjà chargée. Pour une autre police : ouvrir fonts.google.com → choisir la police → copier le code <link> généré dans <head> → utiliser son nom dans font-family. Pas besoin de mémoriser l’URL.

Police — repérer l’HTML avant le CSS

Le sélecteur .font vient directement de class="font".

tuto.html

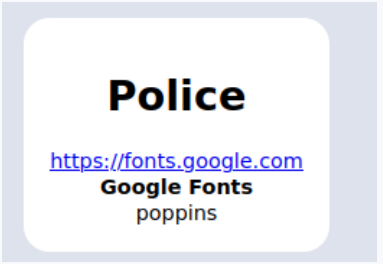
```
<div class="font">
    <h1>Police</h1>
    <a ...>https://fonts.google.com</a>
    <strong>Google Fonts</strong>
    <span>poppins</span>
</div>
```

tuto.css

```
.font * {
    font-family: "Poppins", sans-serif;
}
```

.font → l’élément class="font"

espace * → tous les éléments à l’intérieur



Expérience 1 — changer UNE valeur de font-family

Ne changez ni l'HTML, ni le sélecteur.

AVANT

```
.font * {
    font-family: "Poppins", sans-serif;
}
```

TEST

```
.font * {
    font-family: Arial, sans-serif;
}
```

À FAIRE

1. Dans tuto.css, trouvez .font *
2. Remplacez seulement "Poppins" par Arial
3. Ctrl+S
4. Actualisez le navigateur
5. Regardez UNIQUEMENT le bloc « Police »
6. Testez ensuite "Times New Roman"
7. Remettez "Poppins"

À OBSERVER

La propriété reste font-family ; seule sa valeur change.

Police — ce que l’expérience doit vous faire comprendre

Même HTML, même sélecteur, autre valeur → autre rendu typographique.

Poppins

```
font-family: "Poppins", sans-serif;
```

Arial

```
font-family: Arial, sans-serif;
```

À RETENIR

La police externe est chargée dans <head>. Notre CSS choisit ensuite où et avec quelle valeur l’utiliser.

Material Symbols — pourquoi cette classe affiche une icône ?

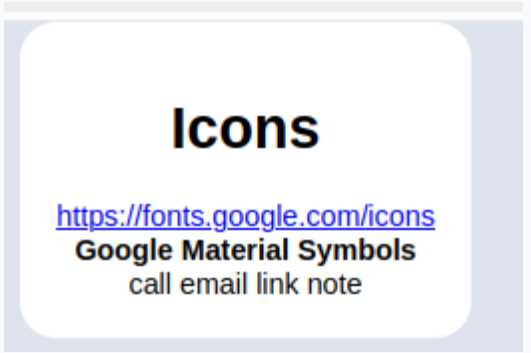
La classe est définie par la feuille de style Google que nous avons importée.

1 — <head> : CHARGER LA BIBLIOTHÈQUE

```
<link rel="stylesheet"
      href="...Material+Symbols+Outlined...">
```

2 — HTML : UTILISER UNE ICÔNE

```
<span class="material-symbols-outlined">
    call
</span>
```



CLASSE DE LA BIBLIOTHÈQUE

material-symbols-outlined
→ nom attendu par Google.

NOM DU SYMBOLE

call / home / email / search…
→ choisit l'icône.

IMPORTANT

Une classe comme « font » peut être nommée par nous. Ici, « material-symbols-outlined » doit garder ce nom pour utiliser la bibliothèque Google. On peut ajouter une classe personnelle en plus.

Material Symbols — choisir une autre icône sur Google

Dans ce support, la bibliothèque est déjà chargée : nous changeons seulement le nom du symbole.

DANS tuto.html

```
<span class="material-symbols-outlined">
    call
</span>
```

DANS CE SUPPORT

Le <link> Material Symbols est DÉJÀ dans <head>.
Vous n’avez rien d’autre à installer pour tester un autre symbole.

À FAIRE SUR LE SITE GOOGLE

1. Dans le bloc « Icons », ouvrez le lien fonts.google.com/icons
2. Recherchez une icône (ex. home, search, delete)
3. Cliquez sur l’icône choisie
4. Repérez / copiez son NOM
5. Revenez dans tuto.html
6. Remplacez seulement call par ce nom
7. Ctrl+S → actualisez

À OBSERVER

La classe reste material-symbols-outlined ; seul le nom entre et change.

Expérience 2 — changer l’icône, puis son apparence

D’abord l’HTML choisit le symbole ; ensuite notre CSS peut le styliser.

HTML — AVANT

```
<span class="material-symbols-outlined">
    call
</span>
```

HTML — TEST

```
<span class="material-symbols-outlined">
    home
</span>
```

À FAIRE — NOM DE L’ICÔNE

1. Trouvez le premier span
2. Remplacez call par home
3. Ctrl+S → actualisez
4. Essayez search puis email
5. Remettez call

OPTION : NOTRE CSS

```
.material-symbols-outlined {
    color: red;
    font-size: 40px;
}
```

À OBSERVER

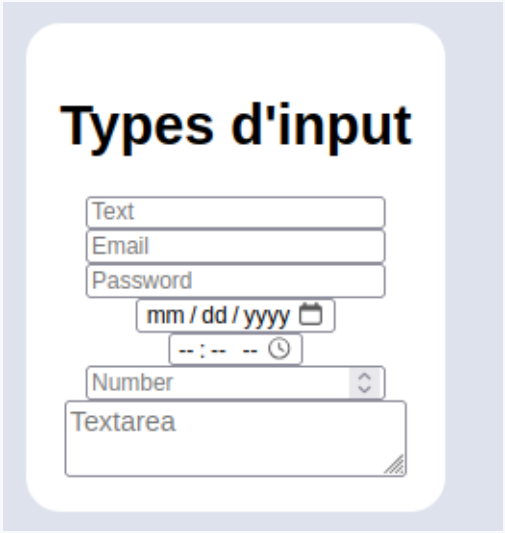
Le nom change le symbole. Notre CSS change son apparence.

Formulaires — un même tag <input>, plusieurs types

Avant de styliser, il faut reconnaître ce qui existe dans l'HTML.

tuto.html

```
<input type="text" ...>
<input type="email" ...>
<input type="password" ...>
<input type="date" ...>
<input type="time" ...>
<input type="number" ...>
<textarea ...></textarea>
```



text	→ texte libre
email	→ adresse e-mail
password	→ caractères masqués
date	→ sélecteur de date
time	→ sélecteur d'heure
number	→ valeur numérique
textarea	→ plusieurs lignes

À RETENIR

type est un attribut HTML standard. text, email, password, date… sont des valeurs standard de cet attribut.

Anatomie d’un <input> — ce qui est standard et ce que nous choisissons

Nous nous concentrons sur les attributs que nous allons réellement réutiliser.

EXEMPLE

```
<input
    type="email"
    name="email"
    id="email"
    placeholder="Email">
```

`input`	**TAG standard**	Le contrôle HTML.
`type`	**ATTRIBUT standard**	Valeur standard : text, email, password…
`id`	**ATTRIBUT standard**	Valeur choisie par nous ; elle doit être unique dans la page.
`name`	**ATTRIBUT standard**	Valeur choisie par nous ; utile lors de l’envoi du formulaire (pas notre focus aujourd’hui).
`placeholder`	**ATTRIBUT standard**	Texte d’aide choisi par nous ; visible quand le champ est vide (ce n’est pas un label).

EXEMPLE IMPORTANT

Les noms type / id / name / placeholder sont standards. type a des valeurs prévues par HTML ; id / name / placeholder ont des valeurs choisies par nous.

placeholder, id, name — qu’est-ce que l’étudiant voit vraiment ?

Les trois sont des attributs HTML standards, mais ils n’ont pas le même rôle.

`placeholder`

VISIBLE quand le champ est vide

HTML

```
<input type="email"
       placeholder="Votre email">
```

Votre email

Puis vous tapez…

tony@example.com

Le placeholder disparaît : ce n’est PAS un label.

`id`

INVISIBLE dans la page

HTML

```
<input id="email" ...>
```

Le mot email est choisi par nous.
Il doit être unique dans la page.

On peut ensuite le retrouver :

CSS

```
#email { ... }
```

`name`

INVISIBLE dans la page

HTML

```
<input name="email" ...>
```

Le mot email est choisi par nous.
Il sert surtout à nommer la donnée quand un formulaire est envoyé.

POUR CE TD

On le reconnaît ; on ne l’étudie pas plus profondément.

Attributs et classes — qu’est-ce qui est standard, qu’est-ce que nous choisissons ?

Le nom de l’attribut vient du HTML ; sa valeur dépend du cas.

EXEMPLE HTML	NOM	VALEUR	POURQUOI ?
`type="email"`	**type = standard HTML**	email = valeur standard HTML	Le navigateur sait quel contrôle créer.
`id="email"`	**id = standard HTML**	email = choisi par nous	La valeur doit être unique dans la page.
`name="email"`	**name = standard HTML**	email = choisi par nous	Nom de la donnée lors de l’envoi.
`placeholder="Email"`	**placeholder = standard HTML**	Email = texte choisi par nous	Aide visible dans un champ vide.
`class="font"`	**class = standard HTML**	font = choisi par nous	On crée .font dans notre CSS.
`class="material-symbols-outlined"`	**class = standard HTML**	valeur attendue par Google	La bibliothèque Google connaît déjà cette classe.

RÈGLE SIMPLE

Les noms viennent du HTML. Les valeurs peuvent être standards, choisies par nous ou imposées par une bibliothèque.

Expérience 3 — changer le type dans l’HTML

Cette fois, nous ne touchons PAS au CSS.

AVANT

```
<input type="text"
       name="text"
       id="text"
       placeholder="Text">
```

TEST TEMPORAIRE

```
<input type="password"
       name="text"
       id="text"
       placeholder="Text">
```

À FAIRE

1. Dans tuto.html, trouvez le premier input
2. Remplacez seulement type="text" par type="password"
3. Ctrl+S → actualisez
4. Écrivez quelques caractères dans ce champ
5. Remettez type="text"

À OBSERVER

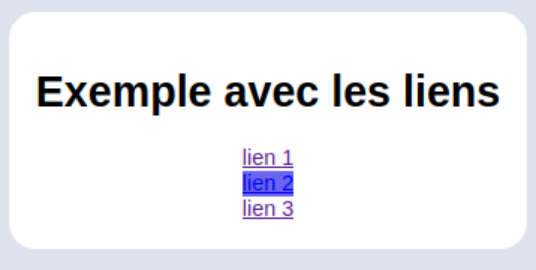
Le comportement du contrôle change alors qu’aucun CSS n’a été modifié.

Sélecteur par attribut — on l’a déjà dans le support

Le bloc « Liens » prépare exactement la syntaxe des inputs.

HTML

```
<a href="#">lien 1</a>
<a href="lien">lien 2</a>
<a href="#">lien 3</a>
```



`a` → tag ciblé

`[href="lien"]`

→ seulement si cet attribut a cette valeur

CSS

```
a[href="lien"] {
    background-color: var(--secondary);
}
```

MÊME IDÉE JUSTE APRÈS

a[href="lien"] → lien précis
input[type="text"] → type d’input précis

Comment cibler seulement le champ Text ?

On construit le sélecteur directement à partir de l'HTML.

HTML

```
<input type="text" ...>
<input type="email" ...>
<input type="password" ...>
```

CSS — SÉLECTEUR

```
input[type="text"]
```

`input` → tag

`[type="text"]` → condition sur l'attribut type

À RETENIR

Le sélecteur n'est pas inventé : regardez l'HTML, choisissez le tag puis l'attribut/valeur qui vous intéresse.

Expérience 4 — décommenter puis changer le sélecteur

Le support CSS est volontairement prêt à tester.

tuto.css — AU DÉPART

```
/* input[type="text"]{
    padding: 10px;
    border: 1px solid #eeeff5;
    border-radius: 5px;
    margin: 10px 0;
} */
```

À FAIRE — TEST 1

1. Enlevez seulement /* et */
2. Ctrl+S → actualisez
3. Comparez Text / Email / Password
4. Quel champ a changé ?

À FAIRE — TEST 2

Remplacez input[type="text"] par input[type="email"] → enregistrez → observez. Essayez ensuite password.

À OBSERVER

Changer le sélecteur change QUI reçoit exactement les mêmes propriétés CSS.

Même style pour plusieurs types — utiliser la virgule

On évite de recopier le même bloc 4 fois.

CSS

```
input[type="text"],
input[type="email"],
input[type="password"],
textarea {
    padding: 10px;
    border: 1px solid #eeeff5;
    border-radius: 5px;
}
```

Virgule = même bloc de propriétés pour plusieurs sélecteurs.

Chaque ligne continue de cibler un élément précis :

- type="text"
- type="email"
- type="password"
- textarea (tag séparé)

À RETENIR

On groupe les contrôles qui doivent avoir la même apparence.

Checkbox + label — la connexion HTML avant le CSS

Le label devient le texte cliquable d’un contrôle précis.

CONTRÔLE

```
<input type="checkbox"
       id="notes">
```

TEXTE ASSOCIÉ

```
<label for="notes">
  Checkbox
</label>
```

CORRESPONDANCE OBLIGATOIRE DANS CETTE ÉCRITURE

id="notes" doit être exactement égal à for="notes"

CE QUI EST STANDARD

id et for sont des attributs HTML standards.
notes est un nom choisi par nous.

EFFET DANS LE NAVIGATEUR

Cliquer sur le mot Checkbox coche ou décoche l’input.

Autre écriture possible : placer l’input à l’intérieur du label. Ici, le support utilise la connexion for ↔ id.

Expérience — prouver que for pointe vers id

Nous cassons volontairement la connexion, puis nous la réparons.

1 — VERSION CORRECTE

```
<input id="notes">
<label for="notes">Checkbox</label>
```

Cliquez sur le texte : la case change.

2 — VERSION CASSÉE

```
<input id="notes">
<label for="autre">Checkbox</label>
```

Cliquez sur le texte : rien ne se passe.

À FAIRE DANS tuto.html

1. Tester for="notes"
2. Remplacer par for="autre"
3. Ctrl+S puis actualiser
4. Tester le mot Checkbox
5. Remettre for="notes"

Aucun CSS n’est nécessaire pour cette connexion : c’est un comportement HTML natif.

CSS — lire input[type="checkbox"]:checked + label

Le sélecteur utilise le type de l'input, son état et sa position dans l'HTML.

1 — QUI ?

input[type="checkbox"]

Un tag input dont type vaut checkbox.

2 — QUAND ?

:checked

Seulement lorsque cette checkbox est cochée.

3 — QUEL ÉLÉMENT ?

+ label

Le label placé immédiatement après l'input.

RÈGLE COMPLÈTE

```
input[type="checkbox"]:checked + label {
  background-color: var(--secondary);
  color: var(--white);
  border-radius: 10px;
}
```

Important : les propriétés entre { } s'appliquent au label, pas à la petite case.

Checkbox — du contrôle natif au label visible

L’HTML conserve l’état ; le CSS décide ce que l’utilisateur voit.

1. Case visible

☐ Checkbox

La checkbox native est visible.
Le label est cliquable grâce à for ↔ id.

2. État checked

Checkbox

:checked devient vrai.
La règle CSS colore le label voisin.

3. display: none

Checkbox

L’input disparaît visuellement.
Son état HTML existe toujours.

4. Label = interface

Checkbox

Cliquer sur le label change encore l’état.
Le label montre cet état.

LES DEUX RÈGLES À RELIER

```
input[type="checkbox"] { display: none; }

input[type="checkbox"]:checked + label { ... }
```

Avant l’exercice — la carte mentale du TD

Tout ce que nous venons de tester se résume à quatre questions.

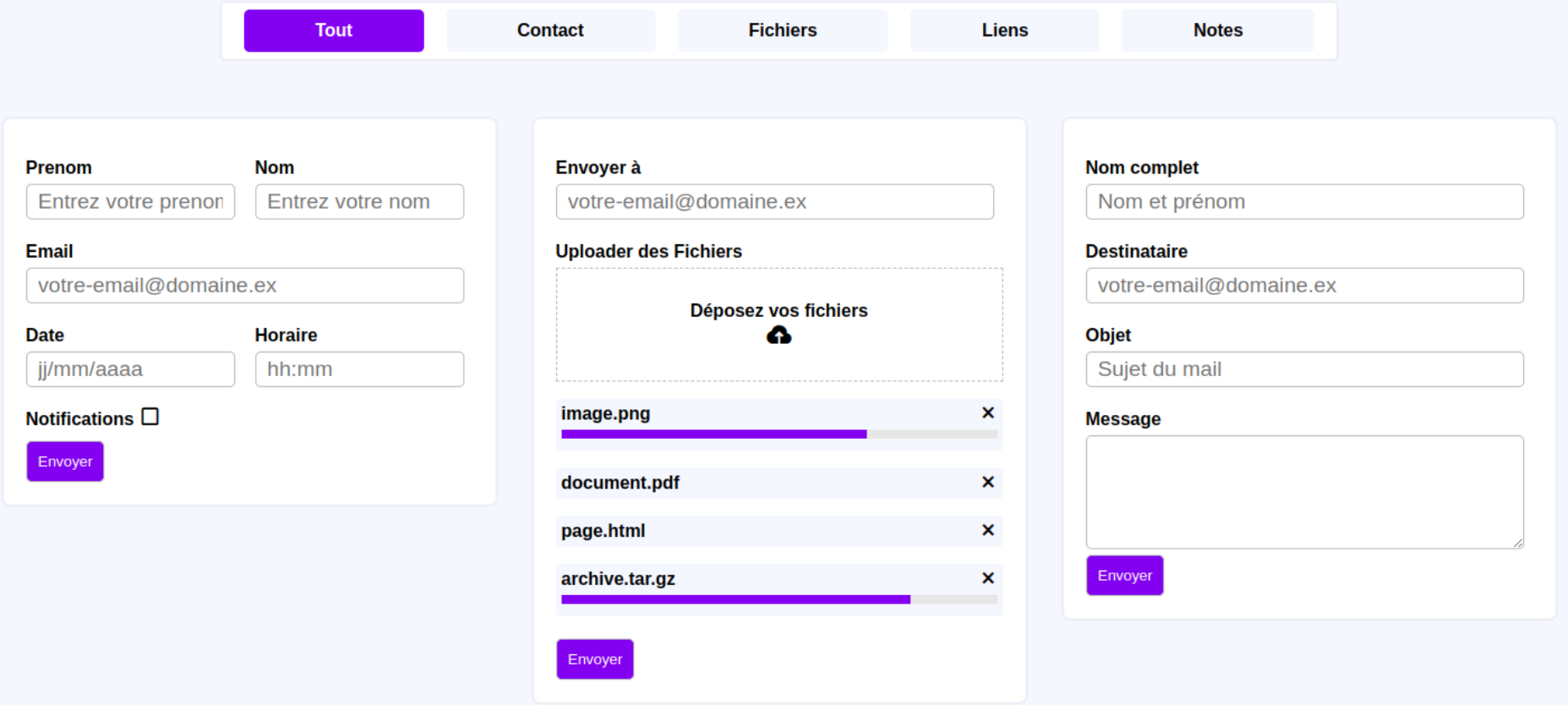
HTML	SÉLECTEUR	ÉTAT	CSS
QU’EST-CE QUI EXISTE ?	QUI JE CIBLE ?	QUAND ?	COMMENT ÇA DOIT PARAÎTRE ?
input / label / textarea / span	`input[type="email"]`	`:checked / :focus / :hover`	border / padding / background…

PLUS LES RESSOURCES EXTERNES

<link> Google Fonts / Material Symbols rendent disponibles des ressources que l’on utilise ensuite dans notre page.

Exercice 1 — objectif final

Nous allons le construire ensemble par couches, sans coller tout le CSS d’un coup.



OBJECTIF

Passer de contrôles HTML standards à cette interface cohérente et interactive.

Exercice — avant le CSS, inspectez l’HTML fourni

Ne commencez pas à écrire tant que vous ne savez pas ce qui existe.

1. `header + checkbox + label`
2. `#content + plusieurs section`
3. `input type="text" / email / date / time`
4. `textarea + button`
5. `input type="file" + .file-upload`
6. `.file + progress`

À FAIRE

Ouvrez l’HTML de l’exercice.
Utilisez Ctrl+F si nécessaire.
Repérez chaque élément de la liste à gauche.
Ne modifiez rien pour l’instant.

POURQUOI ?

Le CSS de l’exercice contient beaucoup de sélecteurs. Si vous savez d’abord ce qui existe dans l’HTML, chaque sélecteur devient beaucoup plus logique.

Exercice — Étape 1A : préparer la page

Nous commençons par body, car il contient le header et #content.

CSS À AJOUTER — commentaires en anglais

```
body {
  --back: #f5f8ff;       /* Page background */
  --white: #ffffff;      /* Card background */
  --secondary: #6a64f1; /* Main accent */


  margin: 0;
  padding: 0;
  background-color: var(--back);


  display: flex;
  flex-direction: column;
  align-items: center;
}
```

OÙ DANS L'HTML ?

<body> contient toute la page :
<header> ... </header>
 ...

POURQUOI ?

Le fond s'applique à toute la page.
Flexbox place ensuite le header et le contenu l'un sous l'autre et les centre.

APRÈS CTRL+S + ACTUALISER

Le fond devient bleu très clair.
Le contenu commence à se centrer.

Exercice — Étape 1B : construire la barre du header

Le sélecteur header cible la zone qui contient les cinq futurs onglets.

CSS DU HEADER

```
header {
  background-color: var(--white);
  height: 50px;
  width: 50%;
  max-width: 1000px;  *maximum width elements allowed to reach*/
  border: 2px solid #eeeff5;
  border-radius: 5px;
  margin: 50px 0;
  padding: 0 20px;


  display: flex;          /* Put items in a row */
  align-items: center;    /* Vertical alignment */
  justify-content: space-around;
  gap: 20px;
}
header > div { flex-grow: 1; }
/* permits the div container to cover the empty space*/
```

REPÉRER AVANT D'ÉCRIRE

Dans l'HTML fourni, cherchez :
<header>
   checkbox + label
  ...
</header>

LIEN HTML → CSS

header cible le conteneur.
header > div cible uniquement ses div enfants directs.

RÉSULTAT ATTENDU

Les cinq contrôles se placent sur une même ligne dans une carte blanche.

Exercice — Étape 1C : placer les trois cartes

#content organise les sections ; section dessine chaque carte.

CSS — CONTENEUR

```
#content {
  display: flex;
  flex-direction: row;
  justify-content: center;
  align-items: flex-start;
  flex-wrap: wrap;
  gap: 30px;
}
```

CSS — CHAQUE CARTE

```
section {
  display: flex;
  flex-direction: column;
  justify-content: center;
  background-color: var(--white);
  border: 2px solid #eeeff5;
  border-radius: 8px;
  padding: 20px;
  width: 400px;
}
```

HTML À RETROUVER

```
<div id="content">
  <section>...</section>
  <section>...</section>
  <section>...</section>
</div>
```

LECTURE

#content place les sections.
section donne le fond, la bordure, l'espace intérieur et la largeur de chaque carte.

APRÈS ACTUALISER

Les trois grands groupes deviennent des cartes séparées avec 30px d'espace.

Exercice — Étape 1D : deux champs sur une ligne

.half-line et .line viennent directement des classes déjà présentes dans l'HTML.

CSS À AJOUTER

```
.half-line {
  width: 100%;
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  flex-wrap: wrap;
}
.half-line > div {
  width: calc(50% - 5px); /* Small gap */
}
.line {
  width: 100%;
  display: flex;
  flex-direction: row;
  align-items: center;
  flex-wrap: wrap;
}
.line, .half-line { margin: 10px 0 5px 0; }
```

HTML À RETROUVER

```
<div class="half-line">
  <div>Prénom...</div>
  <div>Nom...</div>
</div>


<div class="line">Notifications...</div>
```

CE QUE FAIT calc()

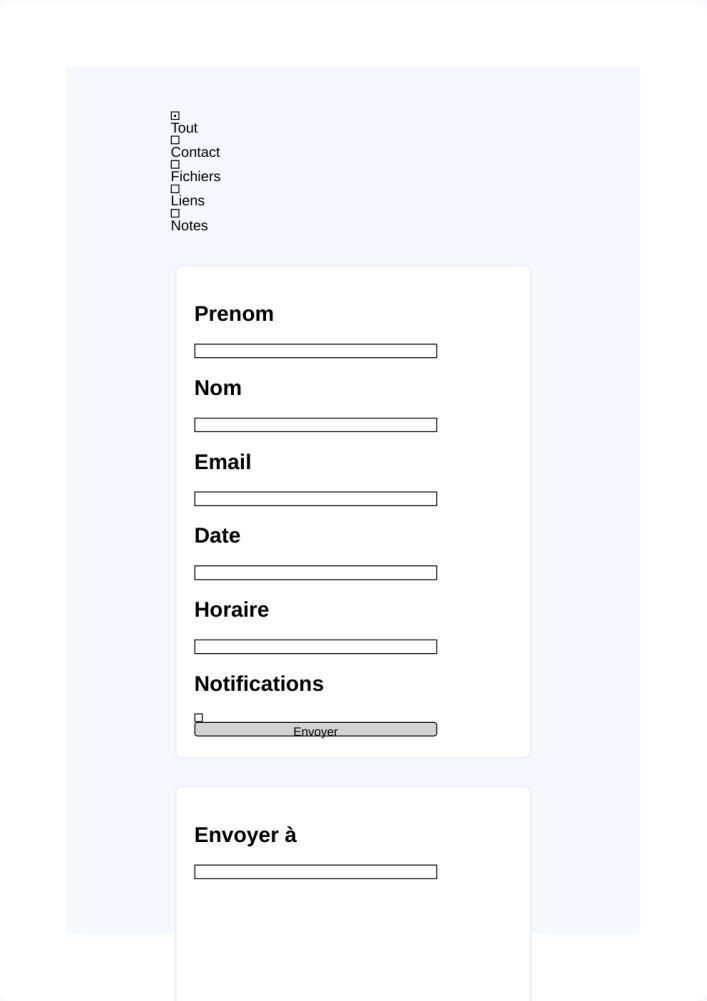
Chaque moitié prend 50% moins 5px.
Les deux blocs tiennent sur la même ligne avec un petit espace.

RÉSULTAT ATTENDU

Prénom/Nom et Date/Horaire apparaissent deux par deux.

Checkpoint 1 — structure de la page

Après body, header, #content, section et les lignes internes.



CE QUI DOIT DÉJÀ ÊTRE VISIBLE

- ✓ fond bleu très clair
- ✓ header en haut
- ✓ cartes blanches
- ✓ espaces entre les cartes
- ✓ certains champs sur deux colonnes

CE QUI RESTE NORMAL À CE STADE

Les checkbox sont encore visibles.
Les labels ne ressemblent pas encore à des onglets.
Les champs utilisent encore leur apparence native.

Exercice — Étape 2A : relire le HTML des onglets

Chaque onglet utilise exactement le mécanisme checkbox + label.

EXTRAIT À REPÉRER DANS LE HTML FOURNI

```
<div>
  <input type="checkbox" id="all" checked>
  <label for="all">Tout</label>
</div>


<div>
  <input type="checkbox" id="contact">
  <label for="contact">Contact</label>
</div>
```

LECTURE

id="all" ↔ for="all"
id="contact" ↔ for="contact"

checked signifie que « Tout » commence coché.

POURQUOI UNE CHECKBOX ?

Elle conserve un état coché/non coché. Le label deviendra l'onglet visible.

AVANT LE CSS DES ONGLETS

La case et son texte sont visibles comme des contrôles HTML ordinaires.

Exercice — Étape 2B : cacher les cases et préparer les labels

L’input garde l’état ; le label prend l’apparence de l’onglet.

CSS À AJOUTER

```
header input[type="checkbox"] {
  display: none; /* Hide native checkbox */
}


header label {
  display: block;
  width: 100%;
  font-weight: bold;
  padding: 10px 0;
  background-color: var(--back);
  border-radius: 5px;
  text-align: center;
  user-select: none; /* Prevent text selection */
  cursor: pointer;   /* Show clickable cursor */
}
```

SÉLECTEUR 1

header input[type="checkbox"]
= les checkbox situées dans header

SÉLECTEUR 2

header label
= les labels situés dans header

APRÈS ACTUALISER

Les petites cases disparaissent.
Les cinq labels deviennent des rectangles cliquables.

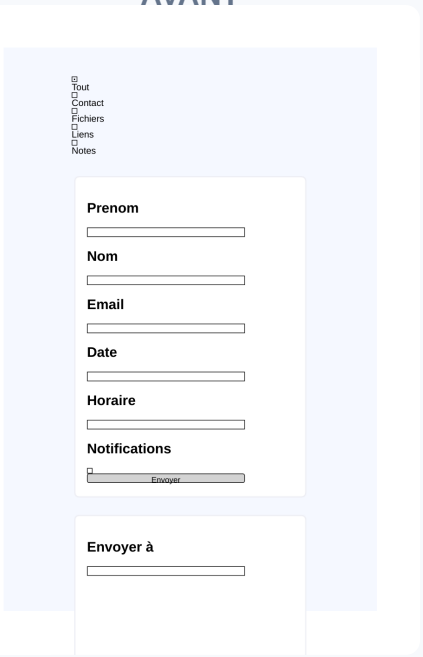
Checkpoint 2 — l’état :checked colore l’onglet

Nous ajoutons une seule règle après avoir préparé les labels.

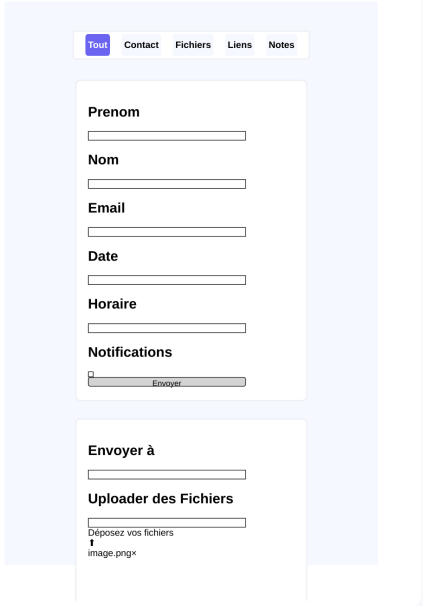
CSS DE L’ÉTAT ACTIF

```
header input[type="checkbox"]:checked + label {
  background-color: var(--secondary);
  color: var(--white);
}
```

AVANT



APRÈS



LECTURE DU SÉLECTEUR

Dans header : une checkbox cochée, puis son label immédiatement après.

Le label reçoit le fond violet et le texte blanc.

TEST

Cliquez sur Contact, puis sur Tout.
Le label cliqué doit changer d’apparence.

Exercice — Étape 3A : uniformiser les champs textuels

Nous regroupons uniquement les contrôles qui doivent partager le même style.

CSS À AJOUTER

```
input[type="text"],
input[type="number"],
input[type="email"],
input[type="password"],
textarea {
  width: calc(100% - 30px);
  padding: 0 10px;
  height: 30px;
  border: 1px solid #bebec0;
  border-radius: 5px;
  font-size: 1.2em;
}
```

LIEN AVEC L'HTML

type="text" → prénom, nom, objet
type="email" → adresses
type="number" / "password" → si présents
textarea → message sur plusieurs lignes

POURQUOI PAS date ET time ?

La correction officielle ne les inclut pas dans ce groupe. Le navigateur garde leur contrôle natif.

APRÈS ACTUALISER

Les champs ciblés prennent la même hauteur, bordure et taille de texte.

Exercice — Étape 3B : textarea et état :focus

Nous séparons la taille du message et l’état actif du champ.

CSS — ZONE MESSAGE

```
textarea {
  height: 100px;
}
```

CSS — CHAMP ACTIF

```
input[type="text"]:focus,
input[type="number"]:focus,
input[type="email"]:focus,
input[type="password"]:focus,
textarea:focus {
  border: 1px solid var(--secondary);
  outline: none; /* Remove browser outline */
}
```

QUE SIGNIFIE :focus ?

Le contrôle est actif pour la saisie : l’utilisateur vient de cliquer dedans ou d’y arriver au clavier.

TEST À FAIRE

Cliquez hors du champ, puis dedans, puis ailleurs. Observez uniquement la bordure.

ATTENTION

outline:none enlève le contour du navigateur. La bordure violette doit rester visible pour signaler le focus.

Checkpoint 3 — champs et boutons

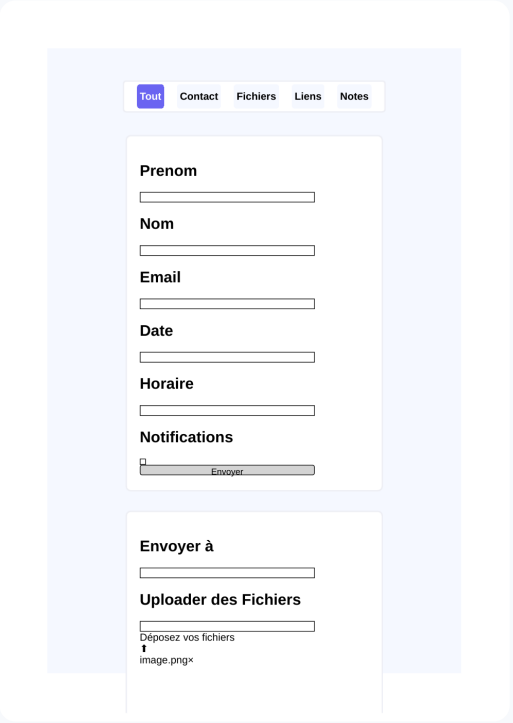
Le formulaire devient lisible avant les checkbox personnalisées et l’upload.

CSS DU BOUTON

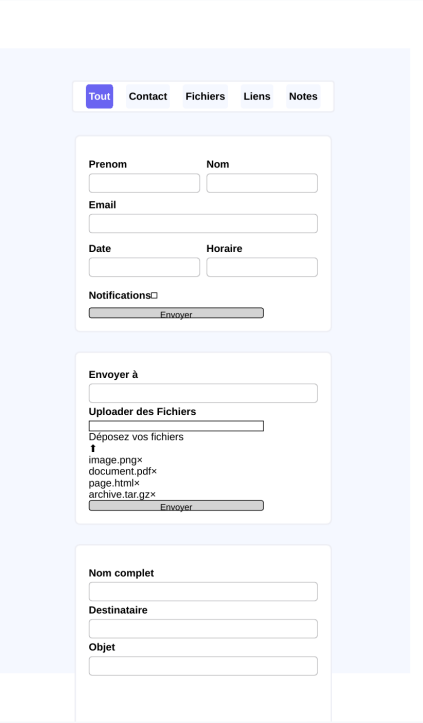
```
button {
  background-color: var(--secondary);
  color: var(--white);
  border: 1px solid #bebec0;
  border-radius: 5px;
  padding: 10px;
  display: inline-block;
  width: fit-content;
}
```

À VÉRIFIER

- ✓ champs alignés
- ✓ textarea plus haute
- ✓ bouton violet
- ✓ bordure focus visible



AVANT



APRÈS

Exercice — Étape 4A : la checkbox Notifications

Même HTML que le tuto, mais limité aux checkbox placées dans section.

HTML À REPÉRER

```
<input type="checkbox" id="notifications">
<label for="notifications"></label>
```

CSS À AJOUTER

```
section input[type="checkbox"] {
  display: none;
}

section input[type="checkbox"] + label {
  user-select: none;
}
```

POURQUOI section ?

Le header utilise aussi des checkbox. section limite cette règle à la notification située dans les cartes.

CONNEXION

for doit toujours correspondre à id. Le label reste cliquable même lorsque l'input est caché.

À CE STADE

La case native disparaît, mais nous n'avons pas encore créé son symbole visible.

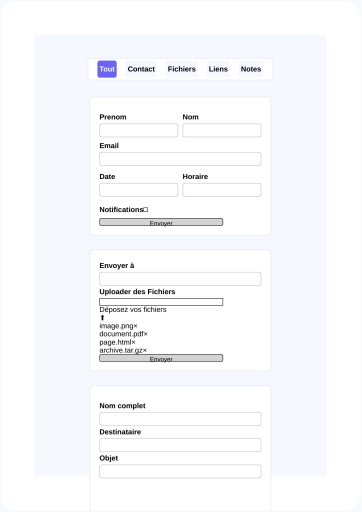
Checkpoint 4 — Material Symbols dessine la case

Cette syntaxe est avancée : nous la lisons sans la mémoriser.

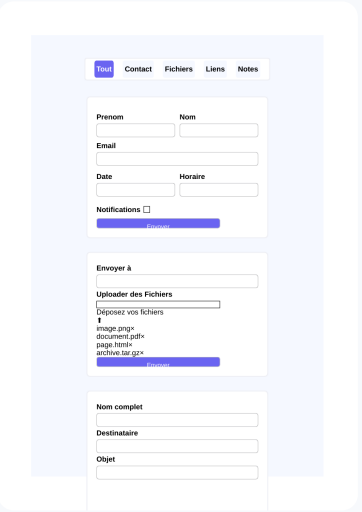
CSS FOURNI DANS LA CORRECTION

```
section input[type="checkbox"] + label:after {
  font-family: "Material Symbols Outlined";
  font-size: 20px;
  line-height: 1;
  content: "check_box_outline_blank";
}


section input[type="checkbox"]:checked + label:after {
  color: var(--secondary);
  content: "check_box";
}
```



AVANT



APRÈS

LECTURE SIMPLE

:after ajoute un contenu après le label.
content choisit le symbole.
:checked remplace le symbole vide par le symbole coché.

Exercice — Étape 5A : input file et label d’upload

Le contrôle natif est caché ; son label devient la zone visible.

HTML À REPÉRER

```
<input type="file" id="upload">
<label class="file-upload" for="upload">
  Déposez vos fichiers
</label>
```

CSS À AJOUTER

```
input[type="file"] {
  display: none; /* Hide native file button */
}


.file-upload {
  width: 100%;
  height: 100px;
  border: 1px dashed #bebec0;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  font-weight: bold;
}
```

MÊME MÉCANISME

for=upload relie le label à id=upload. Cliquer sur la grande zone ouvre donc le sélecteur de fichier.

APRÈS ACTUALISER

Le petit bouton natif disparaît. Une grande zone pointillée et centrée devient l’interface d’upload.

À TESTER

Cliquez au centre de la zone : la boîte de choix de fichier doit s’ouvrir.

Exercice — Étape 5B : présenter chaque fichier

.file dessine une ligne ; .file > div sépare le nom et la croix.

CSS À AJOUTER

```
.file {
  width: calc(100% - 10px);
  background-color: var(--back);
  padding: 5px;
  border-radius: 5px;
  margin-bottom: 15px;
}


.file > div {
  display: flex;
  justify-content: space-between;
}


.file > div > span {
  font-size: 1em;
  font-weight: bold;
}
```

HTML À REPÉRER

```
<div class="file">
  <div>
    <span>image.png</span>
    <span>×</span>
  </div>
  <progress ...></progress>
</div>
```

LECTURE DES SÉLECTEURS

.file > div = le div enfant direct
.file > div > span = les spans à l’intérieur de ce div

APRÈS ACTUALISER

Chaque fichier devient une ligne claire avec le nom à gauche et × à droite.

Checkpoint 5 — barres de progression et résultat complet

La dernière règle essentielle cible le tag progress.

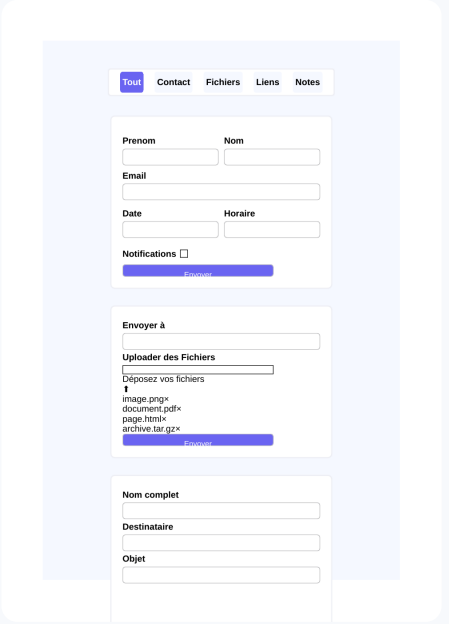
CSS ESSENTIEL

```
progress {
  width: 100%;
  border-radius: 5px;
  margin: 5px 0;
  height: 8px;
}
```

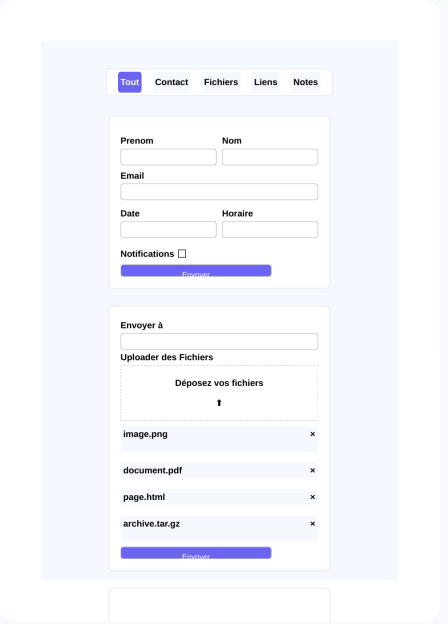
RÈGLES FOURNIES, PAS À MÉMORISER

progress::-webkit-progress-bar
progress::-webkit-progress-value

Elles stylisent des parties internes du progress dans certains navigateurs.



AVANT



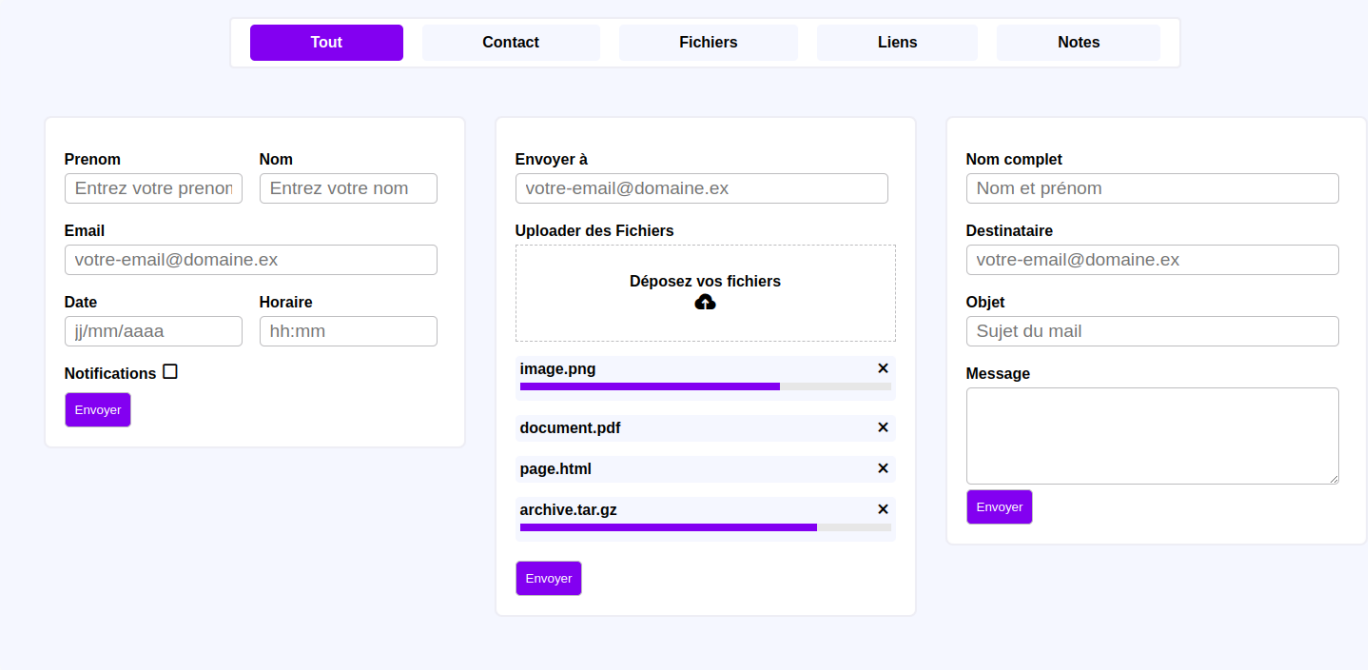
APRÈS

À VÉRIFIER

- ✓ zone pointillée
- ✓ fichiers séparés
- ✓ barres visibles
- ✓ boutons encore cohérents

Exercice — comparaison finale avec la cible officielle

La page est complète ; nous vérifions chaque couche au lieu de relire tout le CSS.



CHECKLIST

- ✓ header horizontal
- ✓ onglet actif violet
- ✓ trois cartes
- ✓ champs cohérents
- ✓ focus visible
- ✓ notification personnalisée
- ✓ zone d’upload
- ✓ fichiers + progression
- ✓ boutons violets

SI UNE PARTIE NE MARCHE PAS

1. Retrouver l’élément dans l’HTML
2. Lire exactement le sélecteur
3. Vérifier propriété et valeur
4. Ctrl+S puis actualiser

TD5 — ce qu’il faut vraiment retenir

Lire la structure, construire le sélecteur, tester une petite modification.

Input natif	type choisit le contrôle ; password masque sans bibliothèque.
Label	for pointe vers l’id du contrôle associé.
Sélecteur d’attribut	input[type="email"] cible un type précis.
État	:checked et :focus ajoutent une condition.
Voisin	+ label cible le label immédiatement après.
Exercice	HTML fourni → petite règle CSS → sauvegarder → observer.

Vous n’avez pas à mémoriser toute la correction. Vous devez savoir expliquer pourquoi chaque règle cible cet élément.